1. **Illustrate the way to represent a string using the C programming language?**

In C, a string is represented as an array of characters ending with a null character '\0'.

Declaration and Initialization:

```
char name[] = "Hello";
```

- Here, name is a character array storing the string "Hello".
- The actual content in memory: {'H', 'e', 'l', 'l', 'o', '\0'}

Using Pointers:

```
char *name = "Hello";
```

Accessing Characters:

```
printf("%c", name[0]);  // Prints 'H'
```

Input/Output of Strings:

```
char str[20];
scanf("%s", str);        // Takes input (stops at space)
printf("%s", str);       // Prints the string
```

2. **Name any five string functions whose prototype is defined in the string.h header file. Demonstrate the working of any two of them.**

Five String Functions from string.h

1. strlen() – Returns the length of a string
2. strcpy() – Copies one string to another
3. strcat() – Concatenates (joins) two strings
4. strcmp() – Compares two strings
5. strrev() – Reverses a string (Note: Not standard in all compilers, but available in some like Turbo C)

1. **strlen() – Get string length**

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello";
    int len = strlen(str);
    printf("Length = %d\n", len);  // Output: Length = 5
    return 0;
}
```

2. **strcpy()– Copy string**

```
#include <stdio.h>
```

```
#include <string.h>
int main() {
    char src[] = "World";
    char dest[20];
    strcpy(dest, src);
    printf("Copied String = %s\n", dest);   // Output: Copied
String = World
    return 0;
}
```

3. **Compose a program in C to multiply two 2D matrices and display the resultant matrix.**

```
#include <stdio.h>
int main() {
    int A[10][10], B[10][10], result[10][10];
    int r1, c1, r2, c2;

    // Input dimensions
    printf("Enter rows and columns for Matrix A: ");
    scanf("%d%d", &r1, &c1);
    printf("Enter rows and columns for Matrix B: ");
    scanf("%d%d", &r2, &c2);

    // Check if multiplication is possible
    if (c1 != r2) {
        printf("Matrix multiplication not possible.\n");
        return 0;
    }

    // Input Matrix A
    printf("Enter elements of Matrix A:\n");
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c1; j++)
            scanf("%d", &A[i][j]);

    // Input Matrix B
    printf("Enter elements of Matrix B:\n");
    for (int i = 0; i < r2; i++)
        for (int j = 0; j < c2; j++)
            scanf("%d", &B[i][j]);

    // Multiply matrices
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < c1; k++)
                result[i][j] += A[i][k] * B[k][j];
        }
    }

    // Display Result
    printf("Resultant Matrix:\n");
```

```
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++)
            printf("%d ", result[i][j]);
        printf("\n");
    }

    return 0;
}
```

4. **Compose a program to input a n x n matrix and print the maximum element of the matrix.**

```
#include <stdio.h>
int main() {
    int matrix[10][10], n, max;

    // Input matrix size
    printf("Enter the size of the matrix (n x n): ");
    scanf("%d", &n);

    // Input matrix elements
    printf("Enter elements of the matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &matrix[i][j]);

    // Assume the first element is the maximum
    max = matrix[0][0];

    // Find the maximum element
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] > max)
                max = matrix[i][j];
        }
    }

    // Output the result
    printf("Maximum element in the matrix is: %d\n", max);

    return 0;
}
```

5. **Create a complete C program to convert a decimal number consisting of an arbitrary number of digits into its binary form using a 1 -D array.**

```
#include <stdio.h>
int main() {
    int decimal, binary[100], i = 0;

    // Input decimal number
    printf("Enter a decimal number: ");
```

```
    scanf("%d", &decimal);

    // Edge case: if input is 0
    if (decimal == 0) {
        printf("Binary: 0\n");
        return 0;
    }

    // Convert to binary and store in array
    while (decimal > 0) {
        binary[i] = decimal % 2;
        decimal = decimal / 2;
        i++;
    }

    // Print binary in reverse
    printf("Binary: ");
    for (int j = i - 1; j >= 0; j--)
        printf("%d", binary[j]);

    printf("\n");
    return 0;
}
```

6. Create a complete C program to find out the Trace of a square matrix [Trace means the sum of Principal Diagonal Elements].

```
#include <stdio.h>
int main() {
    int matrix[10][10], n, trace = 0;

    // Input the size of the square matrix
    printf("Enter the size of the square matrix (n x n): ");
    scanf("%d", &n);

    // Input elements of the matrix
    printf("Enter the elements of the matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &matrix[i][j]);

    // Calculate the trace (sum of principal diagonal elements)
    for (int i = 0; i < n; i++)
        trace += matrix[i][i];

    // Output the trace
    printf("Trace of the matrix = %d\n", trace);

    return 0;
}
```

7. **Compose a C program to find the number of vowels and consonants in a line text.**

```
#include <stdio.h>
#include <ctype.h>  // for tolower()
int main() {
    char line[200];
    int vowels = 0, consonants = 0;

    // Input a line of text
    printf("Enter a line of text:\n");
    fgets(line, sizeof(line), stdin);  // safer input for full lines

    // Loop through each character
    for (int i = 0; line[i] != '\0'; i++) {
        char ch = tolower(line[i]);  // Convert to lowercase for
simplicity

        if ((ch >= 'a' && ch <= 'z')) {  // Check if it is an
alphabet
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' ||
ch == 'u')
                vowels++;
            else
                consonants++;
        }
    }

    // Output the results
    printf("Number of vowels: %d\n", vowels);
    printf("Number of consonants: %d\n", consonants);

    return 0;
}
```

8. **Compose a C program to convert all lower case alphabets to upper case alphabets, a line of text.**

```
#include <stdio.h>
#include <ctype.h>  // for toupper()

int main() {
    char line[200];

    // Input a line of text
    printf("Enter a line of text:\n");
    fgets(line, sizeof(line), stdin);  // reads full line including
spaces

    // Convert lowercase letters to uppercase
    for (int i = 0; line[i] != '\0'; i++) {
        if (islower(line[i])) {
            line[i] = toupper(line[i]);
```

```
        }
    }

    // Output the result
    printf("Converted text: %s", line);

    return 0;
}
```

9. Given a sorted array arr[] of size N and a number X, develop a program in C to find the number of occurrences of X in the given array.

```
#include <stdio.h>

// Function to find the first occurrence of X
int firstOccurrence(int arr[], int n, int x) {
    int low = 0, high = n - 1, result = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == x) {
            result = mid;
            high = mid - 1;  // look on the left side
        } else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return result;
}

// Function to find the last occurrence of X
int lastOccurrence(int arr[], int n, int x) {
    int low = 0, high = n - 1, result = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == x) {
            result = mid;
            low = mid + 1;  // look on the right side
        } else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return result;
}

int main() {
    int arr[100], n, x;

    // Input array size and elements
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    printf("Enter sorted array elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    // Input value to search
    printf("Enter the number to find: ");
    scanf("%d", &x);

    // Find first and last occurrence
    int first = firstOccurrence(arr, n, x);
    int last = lastOccurrence(arr, n, x);

    if (first == -1)
        printf("%d does not occur in the array.\n", x);
    else
        printf("Number of occurrences of %d = %d\n", x, last - first
+ 1);

    return 0;
}
```

10. Build a *C* program to find the union and intersection of two sorted arrays.

```
#include <stdio.h>

// Function to print the union of two sorted arrays
void printUnion(int a[], int b[], int n, int m) {
    int i = 0, j = 0;
    printf("Union: ");
    while (i < n && j < m) {
        if (a[i] < b[j])
            printf("%d ", a[i++]);
        else if (b[j] < a[i])
            printf("%d ", b[j++]);
        else {
            printf("%d ", a[i]);  // equal elements
            i++;
            j++;
        }
    }
    // Remaining elements
    while (i < n) printf("%d ", a[i++]);
    while (j < m) printf("%d ", b[j++]);
    printf("\n");
}

// Function to print the intersection of two sorted arrays
void printIntersection(int a[], int b[], int n, int m) {
    int i = 0, j = 0;
    printf("Intersection: ");
    while (i < n && j < m) {
        if (a[i] < b[j])
```

```
            i++;
        else if (b[j] < a[i])
            j++;
        else {
            printf("%d ", a[i]);  // equal elements
            i++;
            j++;
        }
    }
    printf("\n");
}

int main() {
    int a[100], b[100], n, m;

    // Input first array
    printf("Enter number of elements in first sorted array: ");
    scanf("%d", &n);
    printf("Enter elements of first sorted array:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    // Input second array
    printf("Enter number of elements in second sorted array: ");
    scanf("%d", &m);
    printf("Enter elements of second sorted array:\n");
    for (int i = 0; i < m; i++)
        scanf("%d", &b[i]);

    // Display results
    printUnion(a, b, n, m);
    printIntersection(a, b, n, m);

    return 0;
}
```

11. Represent the impact of an array in programming field.

1. Memory Efficiency and Management: Arrays allow for efficient memory management by storing elements of the same type in contiguous memory locations. This makes accessing elements faster, as the memory address of any element can be calculated easily using the base address and index.

2. Simplified Data Organization and Algorithm Optimization: Arrays provide a structured way to store and access multiple related data items (e.g., a list of numbers, characters). Their simple structure enables efficient implementation of algorithms such as sorting, searching, and traversing, making them fundamental to many programming tasks and optimizing computational efficiency.

12. Each element of an array DATA[15][20] requires 4 bytes of storage. Base address of DATA is 3020. Determine the address of DATA[12][13] when elements are stored in row-major and column major order.

1. **Row-Major Order:**

   Using the formula for row-major order:

   Address(12,13)= 3020 + ((12 x 20) + 13) x 4
   = 3020 + (240 + 13) x 4
   = 3020 + (253 x 4)
   = 3020 + 1012
   = 4032

2. **Column-Major Order:**

   Using the formula for column-major order:

   Address(12,13)= 3020 + ((13 x 15) + 12) x 4
   = 3020 + (195 x 12) x 4
   = 3020 + (207 x 4)
   = 3020 + 828
   = 3848

13. Is it necessary to give a break statement after every switch cases? Explain briefly with appropriate example.

No, it is not strictly necessary to give a break statement after every case in a switch. However, if you do not use break, execution will continue ("fall through") into the next case, even if it does not match.

Example:

```
#include <stdio.h>
int main() {
    int num = 2;
    switch(num) {
        case 1: printf("One\n");
        case 2: printf("Two\n");
        case 3: printf("Three\n");
        default: printf("Default\n");
    }
    return 0;
}
```

Output:

```
Two
Three
Default
```

14. Describe an array? What condition must be satisfied by the entire element of any given array?

An array is a collection of elements of the same data type, stored in contiguous memory locations and accessed using a common name and an index. Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

Condition That Must Be Satisfied by Array Elements:

a. **Same Data Type:** All elements in the array must be of the same type, e.g., all int, all float, or all char.
   Valid: `int arr[3] = {1, 2, 3};`
   Invalid: `int arr[3] = {1, 'A', 3.14};`
b. **Fixed Size at Declaration:** The size of an array is fixed at compile-time (unless using dynamic allocation), and cannot be changed during execution.
c. **Indexed Access:** Each element is accessed by an index, starting from 0 to size-1.

MCQs

1. The output of

```
void main()
{
    int arr[6] = {20, 25};
    printf("%d %d %d", arr[2], arr[3], arr[4]);
}
```

a) 0 0 0

2. What is the output if we consider the following programming code?

```
#include <stdio.h>
int main()
{
    int arr[4] = {1, 2, 3, 4};
    printf("%d", 2[arr]);
    return 0;
}
```

c) 3

3. Which one is the correct output?

```
char a[] = "computer";
printf("%d", strlen(a));
```

c) 8

4. Which of the following is the correct syntax to send an array as a parameter to a C function?

d) Both func(array[size]) and func(*array);

5. Comment on an array of the void data type.

   d) You cannot have an array of void data type

6. Comment on the following 2 arrays with respect to P and Q.

```
int *a1[8];
int *(a2[8]);
```

P. Array of pointers

Q. Pointer to an array

b) a1 is P, a2 is P

7. What are the applications of a multidimensional array?

   a) Matrix-Multiplication

8. What will be the output of the following C code?

```
#include<stdio.h>
int main() {
int ary[2][3][4], j = 20;
ary[0][0] = &j;
printf("%d\n", *ary[0][0]);
}
```

a) Compile time error